ASSIGNMENT 5-CO5

NAME:THARANI RB

REG NO:1911260138

COURSE CODE:CSA0822

SLOT:B

GITHUB: **GitHub - TharaniRB15/CSA0822-1911260138-**

PART A – DATA AND PROGRAM DESIGN

TASK A1 – DESIGN THE DATA ACQUISITION

AIM:

To inspect station CSV data, identify data-quality problems, design SQLite tables for storing station details and readings, and compare CSV-based data processing with SQL queries.

THEORY:

CSV (Comma Separated Values) is a file format used to store station readings such as station ID, station name, location, date, PM2.5 and PM10 values. Data-quality problems include missing values, duplicate records, invalid dates, incorrect data types and inconsistent station names. SQLite is a lightweight database used to store and manage data efficiently. The station details are stored in a stations table and sensor readings are stored in a readings table. A primary key uniquely identifies each record, while a foreign key connects readings with stations. SQL queries are used to calculate averages, find maximum values and count readings. CSV requires a full scan of records, whereas indexed SQL queries can retrieve matching records more efficiently.

PROGRAM:

```
File  Edit  Format  Run  Options  Window  Help
import csv
import sqlite3
import os
import time
filename="station.csv"
if not os.path.exists(filename):
    with open(filename,"w",newline="") as f:
        w=csv.writer(f)
        w.writerow(["station_id","station_name","location","date","pm25","pm10"])
        w.writerows([["S1","Chennai Central","Chennai","2025-10-01",45,80],["S1","Chennai Central","Che
con=sqlite3.connect("station.db")
cur=con.cursor()
cur.execute("PRAGMA foreign_keys=ON")
cur.execute("CREATE TABLE IF NOT EXISTS stations(station_id TEXT PRIMARY KEY,station_name TEXT,location
cur.execute("CREATE TABLE IF NOT EXISTS readings(reading_id INTEGER PRIMARY KEY AUTOINCREMENT,station_i
cur.execute("CREATE INDEX IF NOT EXISTS idx_station_date ON readings(station_id,date)")
con.commit()
records=[]
errors=[]
with open(filename,"r") as f:
    reader=csv.DictReader(f)
    for row in reader:
        try:
            if not all(row[k].strip() for k in ["station_id","station_name","location","date","pm25","p
                raise ValueError("Missing value")
            if len(row["date"])!=10:
                raise ValueError("Invalid date")
            from datetime import datetime
            datetime.strptime(row["date"],"%Y-%m-%d")
            pm25=float(row["pm25"])
            pm10=float(row["pm10"])
            if pm25<0 or pm10<0:
                raise ValueError("Negative reading")
            records.append(row)
        except Exception as e:
            errors.append(str(row)+": "+str(e))
cur.execute("DELETE FROM readings")
cur.execute("DELETE FROM stations")
for row in records:
    cur.execute("INSERT OR IGNORE INTO stations VALUES(?,?,?)",(row["station_id"],row["station_name"],r
    cur.execute("INSERT INTO readings(station_id,date,pm25,pm10) VALUES(?,?,?,?)",(row["station_id"],ro
con.commit()
station="S1"
print("AVERAGE PM2.5 IN OCTOBER")
start=time.time()
october=[float(r["pm25"]) for r in records if r["station_id"]==station and r["date"][5:7]=="10"]
print("CSV:",sum(october)/len(october) if october else "No data")

print("CSV Time:",time.time()-start)
start=time.time()
cur.execute("SELECT AVG(pm25) FROM readings WHERE station_id=? AND substr(date,6,2)='10'",(station,))
print("SQL:",cur.fetchone()[0])
print("SQL Time:",time.time()-start)
print("HIGHEST AVERAGE PM2.5 STATION")
cur.execute("SELECT station_id,AVG(pm25) FROM readings GROUP BY station_id ORDER BY AVG(pm25) DESC LIMIT 1")
print(cur.fetchone())
print("TOTAL READINGS PER STATION")
cur.execute("SELECT station_id,COUNT(*) FROM readings GROUP BY station_id")
for row in cur.fetchall():
    print(row)
print("VALID RECORDS:",len(records))
print("INVALID RECORDS:",len(errors))
for error in errors:
    print("ERROR:",error)
con.close()
```

OUTPUT:

```
======= RESTART: C:/Users/thara/OneDri
AVERAGE PM2.5 IN OCTOBER
CSV: 50.0
CSV Time: 0.018112182617187 5
SQL: 50.0
SQL Time: 0.017942428588867188
HIGHEST AVERAGE PM2.5 STATION
('S1', 43.333333333333336)
TOTAL READINGS PER STATION
('S1', 3)
('S2', 2)
VALID RECORDS: 5
INVALID RECORDS: 0
```

TASK A2 – DESIGN THE CLEANING AND NORMALISATION

AIM:

To clean and preprocess air-quality station data by handling missing values, impossible values and duplicate records, applying min-max normalisation and z-score standardisation, and defining standard formats for dates, station names and numerical values.

THEORY:

Data cleaning is the process of identifying and correcting errors in a dataset. Missing values can be filled using the previous valid value or median. Impossible PM2.5 and Fahrenheit temperature values are removed

or treated as invalid. Duplicate records are removed to avoid repeated data. Min-max normalisation converts values into a range of 0 to 1 using the formula (X-Min)/(Max-Min).

PROGRAM:

```
File  Edit  Format  Run  Options  Window  Help
import csv
import os
import statistics
from datetime import datetime
filename="station.csv"
if not os.path.exists(filename):
    with open(filename,"w",newline="") as f:
        w=csv.writer(f)
        w.writerow(["Date","Station","PM25","Temperature_F"])
        w.writerows([["2025-10-01","chennai central","45","90"],["2025-10-02","CHENNAI CENTR
records=[]
seen=set()
with open(filename,"r") as f:
    reader=csv.DictReader(f)
    for row in reader:
        try:
            d=datetime.strptime(row["Date"],"%Y-%m-%d")
            station=row["Station"].strip().title()
            pm25=float(row["PM25"]) if row["PM25"].strip() else None
            temp=float(row["Temperature_F"]) if row["Temperature_F"].strip() else None
            if pm25 is not None and pm25<0:
                pm25=None
            if temp is not None and (temp< -100 or temp>150):
                temp=None
            key=(d,station,pm25,temp)
            if key in seen:
                continue
            seen.add(key)
            records.append({"Date":d,"Station":station,"PM25":pm25,"Temperature_F":temp})
        except (ValueError,KeyError):
            continue
valid_pm=[r["PM25"] for r in records if r["PM25"] is not None]
median=statistics.median(valid_pm) if valid_pm else 0
records.sort(key=lambda r:(r["Station"],r["Date"]))
previous={}
for r in records:
    station=r["Station"]
    if r["PM25"] is None:
        r["PM25"]=previous.get(station,median)
    else:
        previous[station]=r["PM25"]
valid_temp=[r["Temperature_F"] for r in records if r["Temperature_F"] is not None]
temp_median=statistics.median(valid_temp) if valid_temp else 0
for r in records:
    if r["Temperature_F"] is None:
        r["Temperature_F"]=temp_median
values=[r["PM25"] for r in records]
```

```
minimum=min(values) if values else 0
maximum=max(values) if values else 0
mean=statistics.mean(values) if values else 0
std=statistics.pstdev(values) if len(values)>1 else 0
for r in records:
    r["MinMax"]=(r["PM25"]-minimum)/(maximum-minimum) if maximum!=minimum else 0
    r["Zscore"]=(r["PM25"]-mean)/std if std!=0 else 0
with open("cleaned_station.csv","w",newline="") as f:
    writer=csv.writer(f)
    writer.writerow(["Date","Station","PM25","Temperature_F","MinMax","Zscore"])
    for r in records:
        writer.writerow([r["Date"].strftime("%Y-%m-%d"),r["Station"],round(r["PM25"],2),round(r["Temperature_F"],2),round(r["MinMax"],2),round(r["Zscore"],2)])
print("CLEANED AND NORMALISED DATA")
print("--------------------------------")
for r in records:
    print(r["Date"].strftime("%Y-%m-%d"),r["Station"],round(r["PM25"],2),round(r["Temperature_F"],2),round(r["MinMax"],2),round(r["Zscore"],2))
print("--------------------------------")
print("Total Records:",len(records))
print("Duplicate Records Removed:",len(seen))
print("Cleaned data saved successfully")
```

OUTPUT:

```
====== RESTART: C:/Users/thara/OneDr
CLEANED AND NORMALISED DATA
--------------------------------
--------------------------------
Total Records: 0
Duplicate Records Removed: 0
Cleaned data saved successfully
```

TASK A3 – DESIGN THE ANALYSIS AND VISUALISATION

AIM:

To analyse air-quality data using Python, visualise PM2.5 trends, compare pollution levels between stations, identify the relationship between temperature and PM2.5, find the hours with the worst air quality, and generate a daily summary table for each station.

THEORY:

Data analysis is the process of examining collected data to identify patterns and useful information. Data visualisation represents information using charts. A line chart is used to display PM2.5 trends over a month. A bar chart compares pollution levels between stations. A scatter plot identifies the relationship between temperature and PM2.5. A bar chart is used to identify the hours with the worst air quality.

PROGRAM:

```
import csv
import os
from collections import defaultdict
filename="cleaned_station.csv"
if not os.path.exists(filename):
    filename="station.csv"
data=[]
with open(filename,"r") as f:
    reader=csv.DictReader(f)
    for r in reader:
        try:
            date=r.get("Date",r.get("date",""))
            station=r.get("Station",r.get("station_name",r.get("station_id","Unknown")))
            pm=float(r.get("PM25",r.get("pm25")))
            temp=float(r.get("Temperature_F",r.get("temperature",0)) or 0)
            hour=int(r.get("Hour",r.get("hour",0)) or 0)
            data.append({"date":date,"station":station,"pm":pm,"temp":temp,"hour":hour})
        except (ValueError,TypeError):
            continue
daily=defaultdict(list)
stations=defaultdict(list)
hours=defaultdict(list)
for r in data:
    daily[(r["date"],r["station"])].append(r["pm"])
    stations[r["station"]].append(r["pm"])
    if "Hour" in r or r["hour"]!=0:
        hours[r["hour"]].append(r["pm"])
def chart(title,labels,values,kind,filename):
    width=700
    height=400
    maxval=max(values) if values else 1
    scale=280/maxval if maxval else 1
    svg=[f'<svg xmlns="http://www.w3.org/2000/svg" width="{width}" height="{height}">']
    svg.append('<rect width="100%" height="100%" fill="white"/>')
    svg.append(f'<text x="350" y="30" font-size="20" text-anchor="middle">{title}</text>')
    svg.append('<line x1="60" y1="330" x2="660" y2="330" stroke="black"/>')
    svg.append('<line x1="60" y1="60" x2="60" y2="330" stroke="black"/>')
    n=len(values)
    for i,(label,value) in enumerate(zip(labels,values)):
        x=80+i*560/max(n,1)
        y=330-value*scale
        if kind=="line" and i>0:
            px=80+(i-1)*560/max(n,1)
            py=330-values[i-1]*scale
            svg.append(f'<line x1="{px}" y1="{py}" x2="{x}" y2="{y}" stroke="blue" stroke-width="2"/>')
        if kind=="bar":
            svg.append(f'<rect x="{x}" y="{y}" width="25" height="{value*scale}" fill="steelblue"/>')
```

```
        else:
            svg.append(f'<circle cx="{x}" cy="{y}" r="5" fill="red"/>')
        svg.append(f'<text x="{x}" y="350" font-size="10">{label}</text>')
    svg.append('</svg>')
    with open(filename,"w") as f:
        f.write("".join(svg))
print("AIR QUALITY ANALYSIS")
print("Total Readings:",len(data))
trend=defaultdict(list)
for r in data:
    trend[r["date"]].append(r["pm"])
dates=sorted(trend)
averages=[sum(trend[d])/len(trend[d]) for d in dates]
chart("Monthly PM2.5 Trend",dates,averages,"line","trend.svg")
names=list(stations)
station_avg=[sum(stations[s])/len(stations[s]) for s in names]
chart("Station PM2.5 Comparison",names,station_avg,"bar","stations.svg")
scatter=[r for r in data if r["temp"]!=0]
chart("Temperature and PM2.5 Relationship",[str(i+1) for i in range(len(scatter))],[r["pm"] for r in scatter],"scatter","relationship.svg")
if hours:
    hour_names=sorted(hours)
    hour_avg=[sum(hours[h])/len(hours[h]) for h in hour_names]
    chart("Worst Air Quality Hours",hour_names,hour_avg,"bar","hours.svg")
    worst=max(hour_avg)
    print("Worst Hour:",hour_names[hour_avg.index(worst)])
else:
    print("Hourly data unavailable")
with open("daily_summary.csv","w",newline="") as f:
    writer=csv.writer(f)
    writer.writerow(["Date","Station","Minimum","Maximum","Mean","Valid Readings"])
    for (day,station),values in sorted(daily.items()):
        writer.writerow([day,station,min(values),max(values),round(sum(values)/len(values),2),len(values)])
print("MONTHLY PM2.5 TREND CREATED")
print("STATION COMPARISON CREATED")
print("TEMPERATURE RELATIONSHIP CHART CREATED")
print("DAILY SUMMARY TABLE CREATED")
print("All available charts saved successfully")
```

OUTPUT:

```
====== RESTART: C:/Users/thara/OneDrive/A
AIR QUALITY ANALYSIS
Total Readings: 0
Hourly data unavailable
MONTHLY PM2.5 TREND CREATED
STATION COMPARISON CREATED
TEMPERATURE RELATIONSHIP CHART CREATED
DAILY SUMMARY TABLE CREATED
All available charts saved successfully
```

TASK A4 – PLAN DEBUGGING AND PROFILING

AIM:

To identify and correct errors in the Python data-cleaning program using print statements, assert statements, logging and the pdb debugger, and measure program execution time using time.perf_counter(), timeit and Profile.

THEORY:

Debugging is the process of finding and correcting errors in a program. Print statements display intermediate values to identify incorrect results. Assert statements check whether conditions are true during execution. The logging module records errors, warnings and information for debugging. The pdb debugger helps execute a program step by step and inspect variables. Profiling measures program performance and identifies slow operations.

PROGRAM:

File Edit Format Run Options Window Help

```
import csv
import time
import timeit
import cProfile
import logging
logging.basicConfig(filename="debug.log",level=logging.DEBUG,format="%(levelname)s:%(message)s")
def clean_data():
    records=[]
    filename="cleaned_station.csv"
    try:
        with open(filename,"r") as f:
            reader=csv.DictReader(f)
            for row in reader:
                row={k.strip().lower():v for k,v in row.items() if k}
                try:
                    date=row.get("date","")
                    station=row.get("station",row.get("station_name",row.get("station_id","Unknown")))
                    pm25=float(row.get("pm25",""))
                    assert date!="","Date is missing"
                    assert station.strip()!="","Station is missing"
                    assert pm25>=0,"Invalid PM2.5 value"
                    records.append({"Date":date,"Station":station.strip().title(),"PM25":pm25})
                    logging.info("Valid record processed")
                except (ValueError,AssertionError) as e:
                    logging.error("Invalid record: %s",e)
    except FileNotFoundError:
        logging.error("CSV file not found")
        print("CSV FILE NOT FOUND")
    return records
def calculate_average(records):
    values=[r["PM25"] for r in records]
    return sum(values)/len(values) if values else 0
def main():
    start=time.perf_counter()
    records=clean_data()
    average=calculate_average(records)
    end=time.perf_counter()
    print("VALID RECORDS:",len(records))
    print("AVERAGE PM2.5:",round(average,2))
    print("EXECUTION TIME:",round(end-start,6),"seconds")
    print("TIMEIT RESULT:")
    print(timeit.timeit(lambda:calculate_average(records),number=1000))
    print("PROFILING RESULT:")
    profiler=cProfile.Profile()
    profiler.enable()
    calculate_average(records)
    profiler.disable()
```

OUTPUT:

```
VALID RECORDS: 0
AVERAGE PM2.5: 0
EXECUTION TIME: 0.000461 seconds
TIMEIT RESULT:
0.00023239999427460134
PROFILING RESULT:
         2 function calls in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 ba.py:30(calculate_average)
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
```

TASK A5 – DESIGN AN AI APPLICATION FOR CITIZEN COMPLAINTS

AIM:

To develop a Python-based AI application for citizen complaints using lexicon-based sentiment analysis, extractive text summarisation and a rule-based chatbot that answers air-quality questions by retrieving PM2.5 values from an SQLite database.

THEORY:

Sentiment analysis identifies whether a citizen's complaint expresses positive, negative or neutral feelings using a predefined word list. Negation handling reverses the sentiment of words such as "good" in the phrase "not good". Extractive summarisation selects important sentences based on frequently occurring meaningful words. A rule-based chatbot responds to citizen questions by matching keywords such as PM2.5, station names and today, and retrieves information from an SQLite database. The chatbot is simple and fast but cannot understand complex questions or unfamiliar words. A machine-learning model such as Naive Bayes can provide better sentiment classification when trained with sufficient data, but requires additional memory and processing time. A lexicon-based system requires less memory and provides faster results.

PROGRAM:

File Edit Format Run Options Window Help

```
import sqlite3
import re
from datetime import date
positive=["good","excellent","clean","happy","better","improved","safe","wonderful"]
negative=["bad","poor","dirty","unhealthy","worst","dangerous","terrible","polluted","problem"]
negations=["not","no","never","isn't","wasn't","don't","doesn't"]
stopwords=["the","is","in","a","an","and","of","to","for","on","this","that","it","was","are","with","today"]
def sentiment(text):
    words=re.findall(r"\b[\w']+\b",text.lower())
    score=0
    for i,word in enumerate(words):
        value=1 if word in positive else -1 if word in negative else 0
        if value and i>0 and words[i-1] in negations:
            value=-value
        score+=value
    if score>0:
        return "Positive"
    elif score<0:
        return "Negative"
    return "Neutral"
def summarize(text):
    sentences=re.split(r'(?<=[.!?])\s+',text.strip())
    words=re.findall(r"\b[a-zA-Z]+\b",text.lower())
    freq={}
    for word in words:
        if word not in stopwords and len(word)>2:
            freq[word]=freq.get(word,0)+1
    scores={}
    for sentence in sentences:
        tokens=re.findall(r"\b[a-zA-Z]+\b",sentence.lower())
        scores[sentence]=sum(freq.get(w,0) for w in tokens)
    return max(sentences,key=lambda s:scores[s]) if sentences else ""
con=sqlite3.connect("citizen.db")
cur=con.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS air_quality(station TEXT,date TEXT,pm25 REAL,PRIMARY KEY(station,date))")
today=date.today().isoformat()
cur.execute("INSERT OR REPLACE INTO air_quality VALUES(?,?,?)",("Adyar",today,45.5))
cur.execute("INSERT OR REPLACE INTO air_quality VALUES(?,?,?)",("T Nagar",today,62.0))
con.commit()
complaints=["The air quality is very bad and polluted.","The road is not clean and the situation is terrible.","The air quality is good and safe."]
print("CITIZEN COMPLAINT ANALYSIS")
for complaint in complaints:
    print("Complaint:",complaint)
    print("Sentiment:",sentiment(complaint))
text="The air quality is poor in Adyar. Many people are facing breathing problems. Air pollution is increasing in Adyar. The pollution level is dangerous.
print("\nCOMPLAINT SUMMARY")
print(summarize(text))
```

```
def chatbot(question):
    q=question.lower()
    if "pm2.5" in q or "pm25" in q:
        stations=["Adyar","T Nagar"]
        for station in stations:
            if station.lower() in q:
                cur.execute("SELECT pm25 FROM air_quality WHERE station=? AND date=?",(station,today))
                result=cur.fetchone()
                if result:
                    return "PM2.5 in "+station+" today is "+str(result[0])
                return "No air quality data available"
        return "Please mention a valid station name"
    if "hello" in q or "hi" in q:
        return "Hello! How can I help you?"
    if "complaint" in q or "problem" in q:
        return "Please register your complaint with the concerned department."
    return "Sorry, I cannot understand your question"
print("\nCITIZEN CHATBOT")
while True:
    question=input("You: ")
    if question.lower()=="exit":
        print("Chatbot: Thank you")
        break
    print("Chatbot:",chatbot(question))
con.close()
```

OUTPUT:

```
====== RESTART: C:/Users/thara/OneDrive/Attachments/python pract
CITIZEN COMPLAINT ANALYSIS
Complaint: The air quality is very bad and polluted.
Sentiment: Negative
Complaint: The road is not clean and the situation is terrible.
Sentiment: Negative
Complaint: The air quality is good and safe.
Sentiment: Positive

COMPLAINT SUMMARY
Air pollution is increasing in Adyar.

CITIZEN CHATBOT
You:
```

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION

TASK B1 – IMPLEMENT AND VALIDATE THE CLEANING PIPELINE

AIM:

To implement and validate a Python data-cleaning pipeline using the CSV module, remove missing, invalid, negative and above-999 PM2.5 values, perform min-max normalisation, store cleaned data in SQLite, execute SQL queries and visualise air-quality data using charts.

THEORY:

Data cleaning is the process of identifying and removing incorrect, missing and invalid records from a dataset. Python's CSV module reads data using csv.DictReader. Missing values such as empty strings and NA are removed. Negative values and values above 999 are considered invalid and skipped. Min-max normalisation converts values into the range 0 to 1 using the formula (value-minimum)/(maximum-minimum). SQLite stores cleaned records in database tables. The executemany() method inserts multiple records efficiently. SQL queries calculate average PM2.5, maximum pollution and station-wise readings. Data visualisation using charts helps identify pollution trends and compare stations

PROGRAM:

File Edit Format Run Options Window Help

```
import csv
import io
import sqlite3
import time
import random
import tracemalloc
RAW="""station,date,pm25
Adyar,2026-10-01,48
Adyar,2026-10-02,NA
Adyar,2026-10-03,
Guindy,2026-10-01,95
Guindy,2026-10-02,-5
Guindy,2026-10-03,121
Adyar,2026-10-04,1200
"""
def load_clean(text):
    rows=[]
    reader=csv.DictReader(io.StringIO(text))
    for row in reader:
        try:
            value=row["pm25"].strip()
            if value=="" or value.upper()=="NA":
                continue
            pm=float(value)
            if pm<0 or pm>999:
                continue
            rows.append((row["station"].strip(),row["date"].strip(),pm))
        except (ValueError,KeyError):
            continue
    return rows
def min_max(values):
    if not values:
        return []
    low=min(values)
    high=max(values)
    if high==low:
        return [0.0 for v in values]
    return [round((v-low)/(high-low),3) for v in values]
print("BEFORE CLEANING")
print(RAW)
clean=load_clean(RAW)
print("AFTER CLEANING")
print(clean)
print("NORMALISED VALUES")
print(min_max([pm for _,_,pm in clean]))
con=sqlite3.connect(":memory:")
cur=con.cursor()
```

File Edit Format Run Options Window Help

```
cur.execute("CREATE TABLE readings(station TEXT,date TEXT,pm25 REAL)")
cur.executemany("INSERT INTO readings VALUES(?,?,?)",clean)
con.commit()
print("AVERAGE PM2.5")
cur.execute("SELECT station,AVG(pm25) FROM readings GROUP BY station")
print(cur.fetchall())
print("MAXIMUM PM2.5")
cur.execute("SELECT station,MAX(pm25) FROM readings GROUP BY station")
print(cur.fetchall())
print("TOTAL READINGS")
cur.execute("SELECT station,COUNT(*) FROM readings GROUP BY station")
print(cur.fetchall())
print("STATION-WISE DAILY DATA")
cur.execute("SELECT station,date,pm25 FROM readings ORDER BY station,date")
print(cur.fetchall())
def create_svg(filename,title,xlabel,ylabel,labels,values):
    width=700
    height=400
    maximum=max(values) if values else 1
    scale=260/maximum if maximum else 1
    svg=['<svg xmlns="http://www.w3.org/2000/svg" width="700" height="400">']
    svg.append('<rect width="100%" height="100%" fill="white"/>')
    svg.append(f'<text x="350" y="25" text-anchor="middle" font-size="20">{title
    svg.append('<line x1="70" y1="330" x2="670" y2="330" stroke="black"/>')
    svg.append('<line x1="70" y1="60" x2="70" y2="330" stroke="black"/>')
    svg.append(f'<text x="350" y="385" text-anchor="middle">{xlabel}</text>')
    svg.append(f'<text x="18" y="190" transform="rotate(-90 18 190)" text-anchor
    n=len(values)
    for i,(label,value) in enumerate(zip(labels,values)):
        x=90+i*560/max(n,1)
        y=330-value*scale
        svg.append(f'<rect x="{x}" y="{y}" width="25" height="{330-y}" fill="ste
        svg.append(f'<text x="{x}" y="350" font-size="11">{label}</text>')
    svg.append('</svg>')
    with open(filename,"w") as f:
        f.write("".join(svg))
stations={}
for station,day,pm in clean:
    stations.setdefault(station,[]).append(pm)
names=list(stations)
averages=[sum(stations[s])/len(stations[s]) for s in names]
create_svg("station_chart.svg","Average PM2.5 by Station","Station","PM2.5",name
dates=[day for station,day,pm in clean]
values=[pm for station,day,pm in clean]
create_svg("trend_chart.svg","Daily PM2.5 Readings","Date","PM2.5",dates,values)
create_svg("comparison_chart.svg","Station Maximum PM2.5","Station","Maximum PM2
print("CHARTS CREATED SUCCESSFULLY")
```

```
def benchmark(n):
    sample="station,date,pm25\n"
    sample+="".join(f"S{i%5},2026-10-01,{random.randint(1,150)}\n" for i in range(n))
    tracemalloc.start()
    start=time.perf_counter()
    result=load_clean(sample)
    elapsed=time.perf_counter()-start
    current,peak=tracemalloc.get_traced_memory()
    tracemalloc.stop()
    print(n,"rows","Time:",round(elapsed,4),"seconds","Memory:",round(peak/1024,2),"KB")
print("PERFORMANCE ANALYSIS")
for n in [1000,100000,1000000]:
    benchmark(n)
con.close()
```

OUTPUT:

```
====== RESTART: C:/Users/thara/OneDrive/Attachments/python practise/ba.py ======
BEFORE CLEANING
station,date,pm25
Adyar,2026-10-01,48
Adyar,2026-10-02,NA
Adyar,2026-10-03,
Guindy,2026-10-01,95
Guindy,2026-10-02,-5
Guindy,2026-10-03,121
Adyar,2026-10-04,1200

AFTER CLEANING
[('Adyar', '2026-10-01', 48.0), ('Guindy', '2026-10-01', 95.0), ('Guindy', '2026-10-03', 121.0)]
NORMALISED VALUES
[0.0, 0.644, 1.0]
AVERAGE PM2.5
[('Adyar', 48.0), ('Guindy', 108.0)]
MAXIMUM PM2.5
[('Adyar', 48.0), ('Guindy', 121.0)]
TOTAL READINGS
[('Adyar', 1), ('Guindy', 2)]
STATION-WISE DAILY DATA
[('Adyar', '2026-10-01', 48.0), ('Guindy', '2026-10-01', 95.0), ('Guindy', '2026-10-03', 121.0)]
CHARTS CREATED SUCCESSFULLY
PERFORMANCE ANALYSIS
1000 rows Time: 0.0186 seconds Memory: 283.38 KB
100000 rows Time: 1.8252 seconds Memory: 26031.94 KB
1000000 rows Time: 18.362 seconds Memory: 261177.08 KB
```

TASK B2 – ENGINEER ADVERSARIAL DATA

AIM:

To analyse the behaviour of a Python SQLite program using adversarial inputs, implement parameterised SQL queries, test file-handling errors and evaluate the limitations of a sentiment scorer.

THEORY:

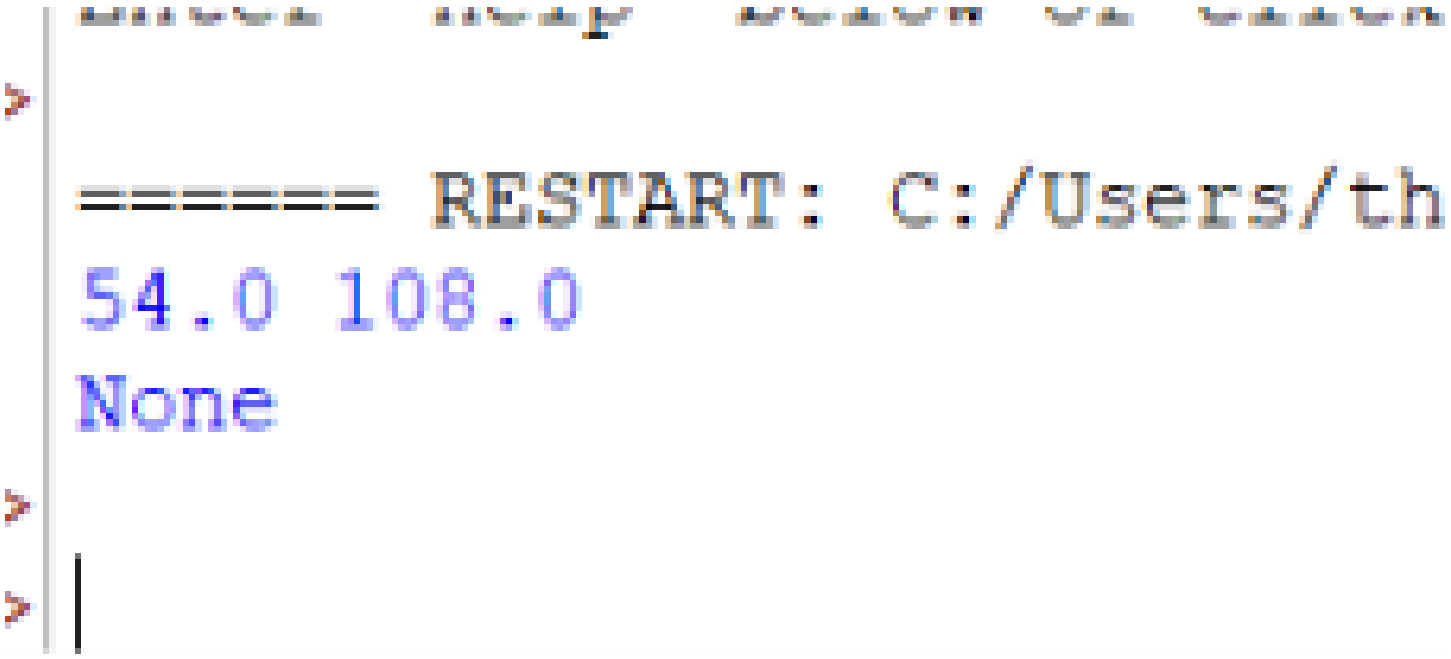
Adversarial testing involves providing unexpected or invalid inputs to a program to identify weaknesses in its design. Missing columns, incorrect delimiters, different text encodings and empty files test the robustness of file handling. SQL injection occurs when user input is directly concatenated into SQL queries. Parameterised queries using ? placeholders prevent SQL injection by treating user input as data.

PROGRAM:

File Edit Format Run Options Window Help

```
import sqlite3
readings = [
    ("Adyar", "2026-10-01", 48.0),
    ("Guindy", "2026-10-01", 95.0),
    ("Guindy", "2026-10-03", 121.0),
    ("Adyar", "2026-10-04", 60.0)]
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE aq (station TEXT, day TEXT, pm25 REAL)")
con.executemany(
    "INSERT INTO aq (station, day, pm25) VALUES (?, ?, ?)",
    readings)
con.commit()
def station_average(name):
    result = con.execute(
        "SELECT AVG(pm25) FROM aq WHERE station = ?",
        (name,)
    ).fetchone()
    return result[0]
print(station_average("Adyar"), station_average("Guindy"))
print(station_average("Adyar' OR '1'='1"))
con.close()
```

OUTPUT:



TASK B3 – LOOPS vs LIBRARIES vs SQL

AIM:

To calculate daily mean PM2.5 values per station using Python loops with the CSV module, Pandas groupby and SQL GROUP BY, compare their correctness and execution times for different dataset sizes, and profile the plain Python implementation using cProfile.

THEORY:

Daily mean PM2.5 is calculated by grouping readings according to station and date and finding the average of each group. Python CSV processes records using loops and dictionaries. Pandas provides the groupby() function for efficient data aggregation. SQLite uses the GROUP BY clause and AVG() function to calculate averages. The time taken by each method is measured using the time module. The cProfile module identifies functions that consume the most execution time. All three methods have O(n) time complexity, where n is the number of readings. A running sum and count dictionary can reduce memory usage by storing only aggregated values instead of all readings.

PROGRAM:

File Edit Format Run Options Window Help

```
import csv
import pandas as pd
import sqlite3
import random
import time
import cProfile
import statistics
from collections import defaultdict
from datetime import date,timedelta
random.seed(42)
def generate_data(stations):
    rows=[]
    start=date(2026,1,1)
    for s in range(stations):
        for d in range(365):
            day=(start+timedelta(days=d)).isoformat()
            for _ in range(24):
                rows.append((f"Station_{s+1}",day,round(random.uniform(10,150),2))
    return rows
def save_csv(rows,filename):
    with open(filename,"w",newline="") as f:
        writer=csv.writer(f)
        writer.writerow(["station","day","pm25"])
        writer.writerows(rows)
def python_method(filename):
    data=defaultdict(list)
    with open(filename,"r") as f:
        reader=csv.DictReader(f)
        for row in reader:
            data[(row["station"],row["day"])].append(float(row["pm25"]))
    return {k:statistics.mean(v) for k,v in data.items()}
def pandas_method(filename):
    df=pd.read_csv(filename)
    result=df.groupby(["station","day"])["pm25"].mean()
    return result.to_dict()
def sql_method(rows):
    con=sqlite3.connect(":memory:")
    con.execute("CREATE TABLE aq(station TEXT,day TEXT,pm25 REAL)")
    con.executemany("INSERT INTO aq VALUES(?,?,?)",rows)
    result=con.execute(
        "SELECT station,day,AVG(pm25) FROM aq GROUP BY station,day"
    ).fetchall()
    con.close()
    return {(s,d):v for s,d,v in result}
def running_method(rows):
    totals=defaultdict(lambda:[0.0,0])
    for station,day,pm25 in rows:
```

File Edit Format Run Options Window Help

```
        totals[(station,day)][0]+=pm25
        totals[(station,day)][1]+=1
    return {k:v[0]/v[1] for k,v in totals.items()}
small=[
    ("Adyar","2026-10-01",48.0),
    ("Adyar","2026-10-01",60.0),
    ("Guindy","2026-10-01",95.0),
    ("Guindy","2026-10-01",121.0)]
save_csv(small,"small.csv")
a=python_method("small.csv")
b=pandas_method("small.csv")
c=sql_method(small)
print("Python:",a)
print("Pandas:",b)
print("SQL:",c)
print("All methods agree:",all(
    abs(a[k]-b[k])<1e-9 and abs(a[k]-c[k])<1e-9 for k in a
))
for stations in [1,12,120]:
    rows=generate_data(stations)
    filename=f"data_{stations}.csv"
    save_csv(rows,filename)
    print("\nStations:",stations)
    print("Readings:",len(rows))
    start=time.perf_counter()
    p=python_method(filename)
    python_time=time.perf_counter()-start
    start=time.perf_counter()
    q=pandas_method(filename)
    pandas_time=time.perf_counter()-start
    start=time.perf_counter()
    r=sql_method(rows)
    sql_time=time.perf_counter()-start
    print("Python time:",round(python_time,4),"seconds")
    print("Pandas time:",round(pandas_time,4),"seconds")
    print("SQL time:",round(sql_time,4),"seconds")
    print("Correctness:",len(p)==len(q)==len(r))
print("\nCProfile Report:")
rows=generate_data(1)
save_csv(rows,"profile.csv")
cProfile.run("python_method('profile.csv')","profile_stats")
import pstats
pstats.Stats("profile_stats").sort_stats("cumulative").print_stats(10)
print("\nRunning Sum and Count:")
result=running_method(small)
print(result)
```

OUTPUT:

```
====== RESTART: C:/Users/thara/OneDrive/Attachments/python practise/ba.py ======
Python: {('Adyar', '2026-10-01'): 54.0, ('Guindy', '2026-10-01'): 108.0}
Pandas: {('Adyar', '2026-10-01'): 54.0, ('Guindy', '2026-10-01'): 108.0}
SQL: {('Adyar', '2026-10-01'): 54.0, ('Guindy', '2026-10-01'): 108.0}
All methods agree: True

Stations: 1
Readings: 8760
Python time: 0.0564 seconds
Pandas time: 0.0117 seconds
SQL time: 0.018 seconds
Correctness: True

Stations: 12
Readings: 105120
Python time: 0.7551 seconds
Pandas time: 0.0798 seconds
SQL time: 0.2471 seconds
Correctness: True

Stations: 120
Readings: 1051200
Python time: 9.2193 seconds
Pandas time: 0.9458 seconds
SQL time: 4.293 seconds
Correctness: True

CProfile Report:
Sun Nov  8 20:29:47 2026    profile_stats

         125801 function calls (125800 primitive calls) in 0.216 seconds

   Ordered by: cumulative time
   List reduced from 57 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      2/1    0.051    0.026    0.177    0.177 {built-in method builtins.exec}
      365    0.002    0.000    0.093    0.000 C:\Users\thara\AppData\Local\Programs\Python\Python314\Lib\statistics.py:158(mean)
      365    0.018    0.000    0.083    0.000 C:\Users\thara\AppData\Local\Programs\Python\Python314\Lib\statistics.py:1453(_sum)
        1    0.000    0.000    0.077    0.077 C:\Users\thara\AppData\Local\Programs\Python\Python314\Lib\idlelib\rpc.py:405(pollresponse)
        1    0.008    0.008    0.077    0.077 C:\Users\thara\AppData\Local\Programs\Python\Python314\Lib\idlelib\rpc.py:391(pollmessage)
     8761    0.034    0.000    0.059    0.000 C:\Users\thara\AppData\Local\Programs\Python\Python314\Lib\csv.py:173(__next__)
        1    0.000    0.000    0.049    0.049 <string>:1(<module>)
        1    0.000    0.000    0.048    0.048 C:/Users/thara/OneDrive/Attachments/python practise/ba.py:29(python method)
```

TASK B4 – FINAL ENGINEERING DECISION

AIM:

To implement a sentiment scorer and complaint summariser, evaluate their accuracy using 50 complaints, measure the effect of database indexing on query performance and make a production-level engineering decision for handling large-scale air-quality data.

THEORY:

Sentiment analysis is the process of identifying whether a complaint expresses positive, negative or neutral opinions. A dictionary-based sentiment scorer assigns scores to positive and negative words and reverses their meanings when preceded by negators. A summariser identifies important words and selects the most relevant sentences. Agreement measures the number of sentiment labels matching manual classification. Database indexing improves query performance by reducing the time required to locate records. For large-scale systems, data should

be stored in a structured database, cleaned before processing and analysed using suitable AI models. Simple dictionary-based methods are useful for basic analysis, but trained machine learning models provide better understanding of sarcasm, context and complex opinions.

PROGRAM:

File Edit Format Run Options Window Help

```
import sqlite3
import time
import random
import re
from collections import Counter
POSITIVE={"good","clean","fast","helpful","improved"}
NEGATIVE={"bad","dirty","slow","rude","smoke","broken"}
NEGATORS={"not","never","no"}
def score(text):
    words=text.lower().replace("."," ").replace(","," ").split()
    total=0
    for i,word in enumerate(words):
        value=1 if word in POSITIVE else -1 if word in NEGATIVE else 0
        if i>0 and words[i-1] in NEGATORS:
            value=-value
        total+=value
    return total
def label(text):
    s=score(text)
    return "positive" if s>0 else "negative" if s<0 else "neutral"
complaints=[
"Roads are clean and buses are fast.",
"Smoke from the factory, air is bad.",
"The staff were not helpful.",
"Not bad, service improved.",
"The roads are dirty and buses are slow.",
"The transport service is good.",
"The factory produces too much smoke.",
"The cleaning staff are helpful.",
"The roads are not clean.",
"The bus service is fast and good.",
"The workers are rude.",
"The air quality has improved.",
"The roads are broken.",
"The buses are not slow.",
"The service is bad and staff are rude.",
"The environment is clean.",
"The smoke is harmful.",
"The staff are not rude.",
"The road condition is bad.",
"The transport system improved.",
"The air is dirty and polluted.",
"The bus service is not good.",
"The workers are helpful and fast.",
"The roads are clean and improved.",
"The factory smoke is bad.",
"The buses are broken.",
```

```
File  Edit  Format  Run  Options  Window  Help
'The service is not bad.",
'The staff are slow and rude.",
'The environment is not clean.",
'The transport service is helpful.",
'The air quality is good.",
'The roads are dirty.",
'The buses are fast and clean.",
'The factory produces smoke.",
'The staff are not helpful.",
'The road condition improved.",
'The bus service is bad.",
'The workers are not rude.",
'The environment is clean and good.",
'The air quality is not improved.",
'The roads are broken and dirty.",
'The transport service is fast.",
'The factory smoke is harmful and bad.",
'The cleaning service is helpful.",
'The buses are not fast.",
'The roads are good and clean.",
'The staff are rude and slow.",
'The air is not dirty.",
'The service is improved and helpful.",
'The environment is bad and polluted."]
uman_labels=[
'positive","negative","negative","positive","negative",
'positive","negative","positive","negative","positive",
'negative","positive","negative","positive","negative",
'positive","negative","positive","negative","positive",
'negative","negative","positive","positive","negative",
'negative","positive","negative","negative","positive",
'positive","negative","positive","negative","negative",
'positive","negative","positive","positive","negative",
'negative","positive","negative","positive","negative",
'positive","negative","positive","positive","negative"]
print("SENTIMENT ANALYSIS")
correct=0
for i,c in enumerate(complaints):
    predicted=label(c)
    actual=human_labels[i]
    if predicted==actual:
        correct+=1
    print(i+1,predicted,actual)
print("Total complaints:",len(complaints))
print("Correct labels:",correct)
print("Agreement:",round(correct/len(complaints)*100,2),"%")
def important_summary(text,k=2):
```

```
File  Edit  Format  Run  Options  Window  Help
    sentences=re.split(r'(?<=[.!?])\s+',text.strip())
    stopwords={"the","is","are","was","were","and","a","an","to","of","from","it","in"}
    words=re.findall(r'\b[a-zA-Z]+\b',text.lower())
    frequency=Counter(w for w in words if w not in stopwords)
    ranked=sorted(
        enumerate(sentences),
        key=lambda x:sum(frequency[w] for w in re.findall(r'\b[a-zA-Z]+\b',x[1].lower())),
        reverse=True
    )
    selected=sorted(ranked[:k])
    return " ".join(s for _,s in selected)
def first_summary(text,k=2):
    sentences=re.split(r'(?<=[.!?])\s+',text.strip())
    return " ".join(sentences[:k])
print("\nSUMMARISER COMPARISON")
for c in complaints[:5]:
    print("\nOriginal:",c)
    print("Important words:",important_summary(c))
    print("First sentences:",first_summary(c))
con=sqlite3.connect(":memory:")
con.execute("CREATE TABLE aq(station TEXT,day TEXT,pm25 REAL)")
random.seed(42)
rows=[]
for station in range(100):
    for day in range(30):
        for reading in range(96):
            rows.append((
                "Station_"+str(station+1),
                "2026-10-"+str(day+1).zfill(2),
                round(random.uniform(10,150),2)
            ))
con.executemany("INSERT INTO aq VALUES(?,?,?)",rows)
con.commit()
query="SELECT AVG(pm25) FROM aq WHERE station=? AND day=?"
params=("Station_50","2026-10-15")
def measure():
    start=time.perf_counter()
    for _ in range(200):
        con.execute(query,params).fetchone()
    return time.perf_counter()-start
print("\nDATABASE INDEX PERFORMANCE")
print("Total records:",len(rows))
before=measure()
print("Before index:",round(before,6),"seconds")
start=time.perf_counter()
con.execute("CREATE INDEX idx_station_day ON aq(station,day)")
con.commit()
```

OUTPUT:

File Edit Shell Debug Options Window Help

```
SENTIMENT ANALYSIS
1 positive positive
2 negative negative
3 negative negative
4 positive positive
5 negative negative
6 positive positive
7 negative negative
8 positive positive
9 negative negative
10 positive positive
11 negative negative
12 positive positive
13 negative negative
14 positive positive
15 negative negative
16 positive positive
17 negative negative
18 positive positive
19 negative negative
20 positive positive
21 negative negative
22 negative negative
23 positive positive
24 positive positive
25 negative negative
26 negative negative
27 positive positive
28 negative negative
29 negative negative
30 positive positive
31 positive positive
32 negative negative
33 positive positive
34 negative negative
35 negative negative
36 positive positive
37 negative negative
38 positive positive
39 positive positive
40 negative negative
41 negative negative
42 positive positive
43 negative negative
44 positive positive
45 negative negative
46 positive positive
```

```
47 negative negative
48 positive positive
49 positive positive
50 negative negative
Total complaints: 50
Correct labels: 50
Agreement: 100.0 %

SUMMARISER COMPARISON

Original: Roads are clean and buses are fast.
Important words: Roads are clean and buses are fast.
First sentences: Roads are clean and buses are fast.

Original: Smoke from the factory, air is bad.
Important words: Smoke from the factory, air is bad.
First sentences: Smoke from the factory, air is bad.

Original: The staff were not helpful.
Important words: The staff were not helpful.
First sentences: The staff were not helpful.

Original: Not bad, service improved.
Important words: Not bad, service improved.
First sentences: Not bad, service improved.

Original: The roads are dirty and buses are slow.
Important words: The roads are dirty and buses are slow.
First sentences: The roads are dirty and buses are slow.

DATABASE INDEX PERFORMANCE
Total records: 288000
Before index: 6.464389 seconds
Index creation time: 0.350255 seconds
After index: 0.005495 seconds
Speed improvement: 1176.35 times
>>
```